

FACULTY OF ENGINEERING
DEPARTMENT OF MECHATRONICS AND BIOMEDICAL
ENGINEERING

END OF SEMESTER 2 COURSEWORK AND
PRESENTATION

Designing a Prepaid Water Credit System

GROUP ONE EVENING:

Name	Registration Number	Signature
1. EPUWAT ELLY BENON	25/U/BIE/18668/PE	
2. AMALI MAUREEN LYDIA	25/U/BIE/05286/PE	
3. MUWONGE RONALD PAUL	25/U/BIE/01401/PE	
4. DEMBE MARK	25/U/BIE/01366/PE	
5. GIZAMBA JOEL	25/U/BIE/01370/PE	

Lecturer:
Mr. OKELLO WAYNE STEVEN

Date: May 26, 2026

Contents

1	Introduction	2
2	System Constants and conditions	2
3	Declaration of functions	2
4	The use of structures.	3
5	Mathematical Business Logic	3
5.1	Calculation of gross amount	3
5.2	Calculating number of credit units	3
6	Main requirements of the program.	3
7	Source Code Implementation	5
8	Conclusion	10

1 Introduction

The system is a program designed to manage utility consumption in this case water consumption for either domestic or commercial meters. The system is called "HAVEN PREPAID WATER SYSTEM". It enter the amount of money paid by a customer or user and calculates the number of units for the amount paid. The program features structured security via system token authentications, records management (that is; Create, Read, Update, Delete) for up to 50 distinct household records.

2 System Constants and conditions

The program required authentication and some fixed values for calculations for example; maintenance fee, charge for domestic meter and commercial meter and the alert safety limit. these values are listed below.

- **System Access PIN:** 4545
- **Fixed Maintenance Fee Deducted:** 500.0 UGX
- **Domestic Unit Rate:** 150.0 UGX per Unit
- **Commercial Unit Rate:** 250.0 UGX per Unit
- **Leakage Alert Safety Limit:** 50.0 Units per day

3 Declaration of functions

- `int authenticate();` – Validates user access by requesting for the system security PIN.
- `void addHousehold();` – Allocates storage and activates a new customer record profile.
- `void searchHousehold();` – Looks through the existing system registry to locate a targeted household profile.
- `void updateHousehold();` – Modifies parameters or utility variables within an existing record.
- `void deleteHousehold();` – Permanently removes a consumer record from the active memory of the program.
- `void buyWaterCredit();` – Evaluates transaction inputs, applies deductions, and prints tokenized units.
- `void displaySummary();` – Compiles and prints comprehensive billing and usage registries.

4 The use of structures.

Structure are basically used to store data. Unlike arrays structures store data in different data types and can store many streams of data for one structure. The structure configuration is shown below.

```
typedef struct {
    int id;
    char headName[50];
    char type[15];
    float balanceUnits;
    float dailyUsageUnits;
} Household;
```

5 Mathematical Business Logic

The program works in a way that when a user or household owner makes a deposit(money),M, it deducts a maintenance fee,F_m of 500UGX and the balance(grossAmount),M_{net} is used to calculate the units bought by dividing by the dynamicRate(for either domestic or commercial meter). The mathematical evaluation is modeled below:

5.1 Calculation of gross amount

Given a gross money M , the processing grossAmount M_{net} is computed after a fixed maintenance fee (F_m) extraction:

$$M_{net} = M - F_m \quad (1)$$

If $M \leq F_m$, the operational routine flags an immediate structural evaluation error and terminates execution.

5.2 Calculating number of credit units

The final number of credit units (ΔU) are scaled dynamically based on a string mapping logical condition checking account categories:

$$\Delta U = \frac{M_{net}}{\text{Tariff Rate}} \quad (2)$$

Where:

$$\text{Tariff Rate} = \begin{cases} 250.0 \text{ UGX,} & \text{if Type} = \text{"Commercial"} \\ 150.0 \text{ UGX,} & \text{if Type} = \text{"Domestic"} \end{cases} \quad (3)$$

6 Main requirements of the program.

- To receive household information and store it using structures.
- To be able to edit, update and delete the household information.
- To calculate the number of credit units depending on how much a user has input.

- The program should print a receipt after the calculation of the credit units showing the units purchased and the balance of units.
- It should be able to print the summary of records of households stored in the system showing all their details including credit units and balance.
- The program should be able to detect leakage and low credit balance and display an appropriate error or warning message.

7 Source Code Implementation

The program runs on C program implementing majority of the functions and C programming features including libraries like the standard input output library, `stdio.h` and the string library, `string.h`. Other features used were structures, loops, the if conditions, whiles and do-while among others.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX_HOUSEHOLDS 50
5 #define AUTH_PIN "4545"           // System access PIN for
   authentication
6 #define MAINTENANCE_FEE 500.0     // Fixed maintenance fee
7 #define DOMESTIC_RATE 150.0      // Cost per unit for Domestic
8 #define COMMERCIAL_RATE 250.0    // Cost per unit for Commercial
9 #define LOW_CREDIT_LIMIT 5.0      // Low credit threshold in units
10 #define LEAKAGE_LIMIT 50.0       // Unusual high daily usage
   threshold
11
12 // Structure to hold household records
13 typedef struct {
14     int id;
15     char headName[50];
16     char type[15]; // "Domestic" or "Commercial"
17     float balanceUnits;
18     float dailyUsageUnits;
19 } Household;
20
21 // Global variables
22 Household records[MAX_HOUSEHOLDS];
23 int householdCount = 0;
24
25 // Function Prototypes
26 int authenticate();
27 void menu();
28 void addHousehold();
29 void searchHousehold();
30 void updateHousehold();
31 void deleteHousehold();
32 void buyWaterCredit();
33 void displaySummary();
34
35 int main() {
36     printf("=== HAVEN PREPAID WATER SYSTEM ===\n");
37     if (!authenticate()) {
38         printf("Authentication failed! Exiting system.\n");
39         return 0;
40     }
41
42     // Seed sample data for easy testing
```

```

43     records[0] = (Household){101, "Mark Dembe", "Domestic", 12.5,
44     8.0};
45     records[1] = (Household){102, "Prosper Ltd", "Commercial",
46     3679.0, 55.0};
47     householdCount = 2;
48
49     menu();
50     return 0;
51 }
52
53 // Simple authentication rule integration
54 int authenticate() {
55     char pin[10];
56     printf("Enter System Access PIN: ");
57     scanf("%s", pin);
58     if (strcmp(pin, AUTH_PIN) == 0) return 1;
59     return 0;
60 }
61
62 // Main operational loop
63 void menu() {
64     int choice;
65     do {
66         printf("\n*****\n");
67         printf("                MAIN MENU                \n");
68         printf("*****\n");
69         printf("1. Register New Household\n");
70         printf("2. Search Household Record\n");
71         printf("3. Update Household Details\n");
72         printf("4. Delete Household Record\n");
73         printf("5. Buy Water Credit (Token)\n");
74         printf("6. Check for the summary of records\n");
75         printf("7. Exit System\n");
76         printf("Enter choice (1-7): ");
77         scanf("%d", &choice);
78
79         switch(choice) {
80             case 1: addHousehold(); break;
81             case 2: searchHousehold(); break;
82             case 3: updateHousehold(); break;
83             case 4: deleteHousehold(); break;
84             case 5: buyWaterCredit(); break;
85             case 6: displaySummary(); break;
86             case 7: printf("\nThanks for using Haven Systems!\n");
87                     break;
88             default: printf("Invalid option! Please try again.\n");
89         }
90     } while(choice != 7);
91 }

```

```

90 // 1. Create Operation
91 void addHousehold() {
92     if (householdCount >= MAX_HOUSEHOLDS) {
93         printf("Database full! Cannot register more households.\n
94     ");
95         return;
96     }
97     Household h;
98     printf("\nEnter Unique Household ID (Integer): ");
99     scanf("%d", &h.id);
100
101     // Check if ID already exists
102     for(int i=0; i<householdCount; i++) {
103         if(records[i].id == h.id) {
104             printf("Error: A household with ID %d already exists
105         .\n", h.id);
106             return;
107         }
108     }
109
110     printf("Enter Househead Name: ");
111     scanf(" %[^\n]s", h.headName);
112
113     int typeChoice;
114     printf("Select Type (1. Domestic, 2. Commercial): ");
115     scanf("%d", &typeChoice);
116     if(typeChoice == 2) strcpy(h.type, "Commercial");
117     else strcpy(h.type, "Domestic");
118
119     h.balanceUnits = 0.0; // Initial balance starts at 0
120     h.dailyUsageUnits = 0.0;
121
122     records[householdCount++] = h;
123     printf("Household registered successfully!\n");
124 }
125
126 // 2. Read / Search Operation
127 void searchHousehold() {
128     int id, found = 0;
129     printf("\nEnter Household ID to search: ");
130     scanf("%d", &id);
131
132     for (int i = 0; i < householdCount; i++) {
133         if (records[i].id == id) {
134             printf("\n--- Record Found ---\n");
135             printf("ID: %d\nName: %s\nType: %s\nBalance Units:
136             %.2f\nDaily Usage: %.2f units\n",
137                 records[i].id, records[i].headName, records[i
138             ].type, records[i].balanceUnits, records[i].dailyUsageUnits);
139
140             // Warnings logic processing

```



```

137         if(records[i].balanceUnits < LOW_CREDIT_LIMIT) {
138             printf("[WARNING]: Low water credit! Please top
up soon.\n");
139         }
140         if(records[i].dailyUsageUnits > LEAKAGE_LIMIT) {
141             printf("[CRITICAL ALERT]: High daily usage
detected! Possible water leakage.\n");
142         }
143         found = 1;
144         break;
145     }
146 }
147 if (!found) printf("Household record not found.\n");
148 }
149
150 // 3. Update Operation
151 void updateHousehold() {
152     int id, found = 0;
153     printf("\nEnter Household ID to update: ");
154     scanf("%d", &id);
155
156     for (int i = 0; i < householdCount; i++) {
157         if (records[i].id == id) {
158             printf("Enter New Name for Househead: ");
159             scanf(" %[^\\n]s", records[i].headName);
160             printf("Enter Current Daily Water Usage (units): ");
161             scanf("%f", &records[i].dailyUsageUnits);
162             printf("Record updated successfully!\n");
163             found = 1;
164             break;
165         }
166     }
167     if (!found) printf("Record not found.\n");
168 }
169
170 // 4. Delete Operation
171 void deleteHousehold() {
172     int id, found = 0;
173     printf("\nEnter Household ID to delete: ");
174     scanf("%d", &id);
175
176     for (int i = 0; i < householdCount; i++) {
177         if (records[i].id == id) {
178             found = 1;
179             for (int j = i; j < householdCount - 1; j++) {
180                 records[j] = records[j + 1];
181             }
182             householdCount--;
183             printf("Household record deleted successfully!\n");
184             break;
185         }

```

```

186     }
187     if (!found) printf("Record not found.\n");
188 }
189
190 // 5. Core Utility Rule Business Logic Processing
191 void buyWaterCredit() {
192     int id, found = 0;
193     float money, dynamicRate, grossAmount, unitsBought;
194
195     printf("\nEnter Household ID for token purchase: ");
196     scanf("%d", &id);
197
198     for (int i = 0; i < householdCount; i++) {
199         if (records[i].id == id) {
200             found = 1;
201             printf("Enter amount of money to deposit (UGX): ");
202             scanf("%f", &money);
203
204             // Validation Rule: Check if money can cover the
205             maintenance fee
206             if (money <= MAINTENANCE_FEE) {
207                 printf("[ERROR]: Amount entered is too low to
208                 process after a maintenance deduction of %.0f UGX.\n",
209                 MAINTENANCE_FEE);
210                 return;
211             }
212
213             grossAmount = money - MAINTENANCE_FEE;
214
215             // Tiered pricing rate application
216             if (strcmp(records[i].type, "Commercial") == 0) {
217                 dynamicRate = COMMERCIAL_RATE;
218             } else {
219                 dynamicRate = DOMESTIC_RATE;
220             }
221
222             unitsBought = grossAmount / dynamicRate;
223             records[i].balanceUnits += unitsBought;
224
225             // Purchase Summary Printout Requirement
226             printf("\n***** WATER CREDIT RECEIPT *****\n");
227             printf("Household ID:    %d\n", records[i].id);
228             printf("Amount Paid:    %.2f UGX\n", money);
229             printf("Maintenance Fee: %.2f UGX\n", MAINTENANCE_FEE);
230
231             ;
232             printf("Rate per Unit:    %.2f UGX\n", dynamicRate);
233             printf("Units Bought:    %.2f units\n", unitsBought);
234             printf("Updated Balance: %.2f units\n", records[i].
235             balanceUnits);
236             printf("*****\n");
237             break;

```

```

232     }
233 }
234     if (!found) printf("Household record not found.\n");
235 }
236
237 // 6. Report Generator View
238 void displaySummary() {
239     printf("\n
240 *****\n");
241     printf("
242             ALL HOUSEHOLD WATER LOG RECORDS
243     \n");
244     printf("
245 *****\n"
246 );
247     printf("%-6s | %-15s | %-12s | %-10s | %-10s\n", "ID", "
248 Househead", "Type", "Balance", "Daily Use");
249     printf("
250 -----\n");
251     for (int i = 0; i < householdCount; i++) {
252         printf("%-6d | %-15s | %-12s | %-10.2f | %-10.2f\n",
253             records[i].id, records[i].headName, records[i].
254 type, records[i].balanceUnits, records[i].dailyUsageUnits);
255     }
256     printf("
257 *****\n"
258 );
259 }

```

Listing 1: Haven Prepaid Water System - Source Implementation

8 Conclusion

The application of code and the use of c programming provided and efficient tool to automate the calculation of water utility consumption. By replacing manual billing methods, the software minimizes human error and significantly speeds up data processing. The system also prints and displays precise and summarized data from transactions like receipts and summary tables of stored information. The program also has room for further modification and improvement into better versions and could also be modified for other operations and systems in the future.